

BÀI THỰC HÀNH SỐ 3

TÌM HIỂU KIẾN TRÚC CỦA BỘ DMA (DIRECT MEMORY ACCESS)

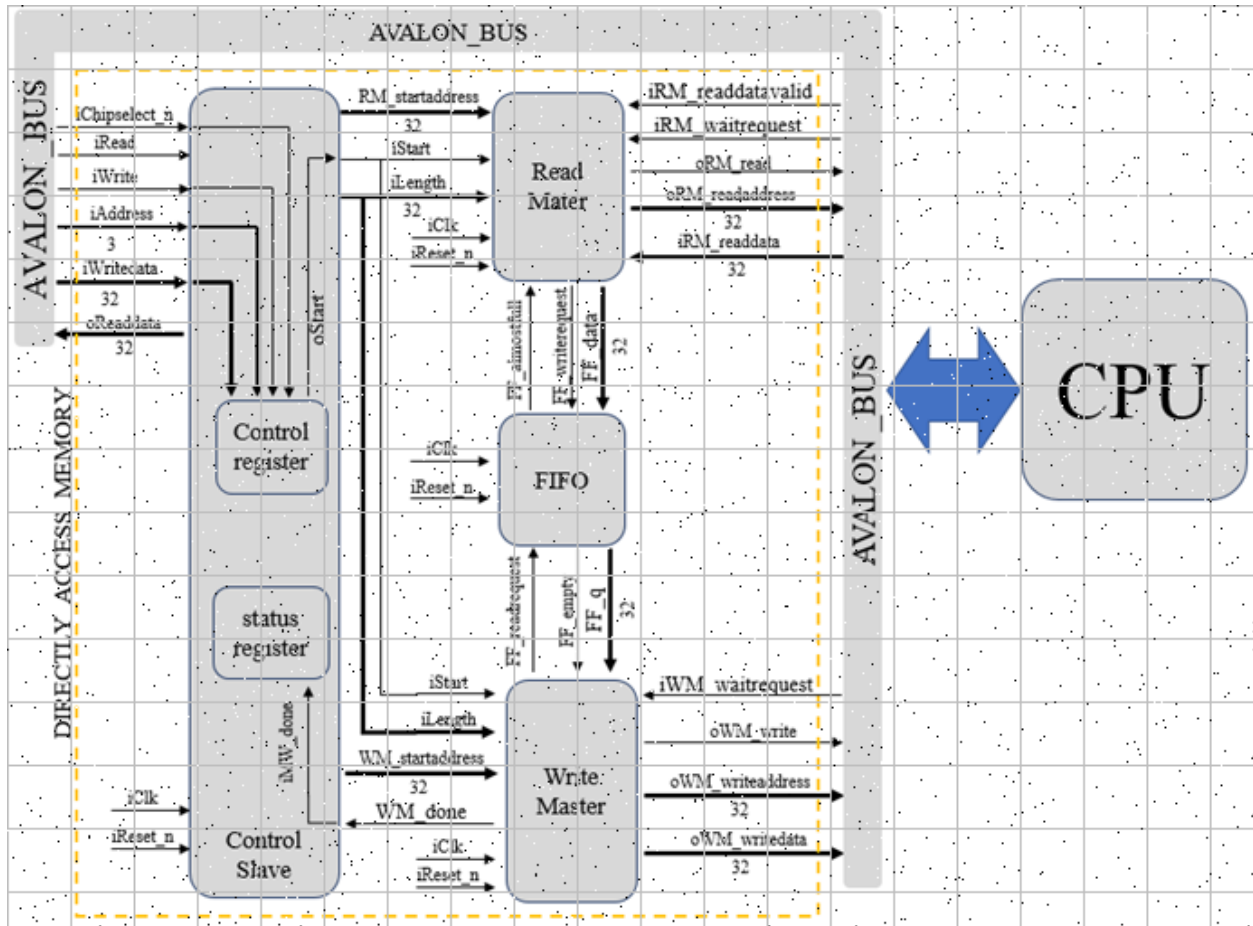
Yêu cầu:

- Sinh viên hiểu được kiến trúc DMA với một số tín hiệu cơ bản, giải thích được các tín hiệu trong code, thanh ghi config và cơ chế hoạt động của DMA
- Sinh viên dùng source code được cung cấp để mô phỏng lại bộ DMA trên phần mềm modelsim (chụp kết quả phải thấy ngày tháng năm và giờ phút giây)
- Điểm thực hiện cho bài lab này 8 điểm nếu file báo cáo chỉ dừng lại với 2 yêu cầu trên. 10 điểm nếu áp dụng Signal TapII bắt được tín hiệu DONE của thanh ghi config (dùng KIT DE2)

Việc thiết kế bộ DMA (direct access memory) sẽ chia thành 2 phần: Thiết kế (viết) các thành phần của DMA sử dụng ngôn ngữ verilog và Nhúng các thành phần này vào trong qsys để có thể sử dụng

1. Thiết kế các thành phần của DMA sử dụng ngôn ngữ verilog

Bộ DMA gồm những module cơ bản: FIFO, read master, write master và control slave.



Hình 1.1. Sơ đồ khối của DMA controller

1.1. Thiết kế module FIFO

- **Lý thuyết:** dung lượng của FIFO sẽ gồm 256 thang ghi 32bit (1028 bytes), gồm có các ngõ kết nối như sau:
 - Tín hiệu vào xung clock (**iClk**)
 - Tín hiệu vào reset tích cực ở mức thấp (**iReset_n**)
 - Tín hiệu ra báo FIFO đang trống (**FF_empty**) cho write master/ read master

- Tín hiệu ra báo FIFO gần đầy (**FF_almostfull**) cho read master/ write master
- Tín hiệu vào yêu cầu đọc (**FF_readrequest**)
- Tín hiệu vào yêu cầu ghi (**FF_writerequest**)
- Dữ liệu ngõ vào bus 32bit (**FF_data**)
- Dữ liệu ngõ ra bus 32bit (**FF_q**)

Trong file thiết kế sẽ sử dụng 2 con trỏ vị trí để xác định rằng FIFO đang trống hay gần đầy (*reg [7:0] pos_read, pos_write*, dùng thanh ghi 8bit ([7:0]) là vì sử dụng 256 thanh ghi 32bit, do đó cần thanh ghi 8bit để lưu giá trị địa chỉ tương ứng cho chúng).

Trước khi tiến hành đọc hay ghi dữ liệu đối với FIFO, sẽ thực hiện thao tác để tiến hành ghi và đọc tương ứng:

```
assign fifo_wr = (~FF_almostfull) & FF_writerequest;
assign fifo_rd = (~FF_empty & FF_readrequest);
```

Tiếp theo, nếu tín hiệu reset được cho ở mức tích cực, vị trí con trỏ đọc/ghi (*pos_read/ pos_write*) sẽ bằng 0, nếu không, nếu **fifo_wr** = 1/ **fifo_rd** = 1 sẽ ghi dữ liệu vào FIFO/đọc dữ liệu từ FIFO ra, sau đó, tăng con trỏ *pos_write/pos_read* lên 1 đơn vị.

Cuối cùng, so sánh bit cao nhất của *pos_read* và *pos_write* bằng dòng code:

```
assign compare = pos_write [7] ^ pos_read [7];
```

tiếp theo kiểm tra vị trí của 2 con trỏ có bằng nhau không bằng dòng code:

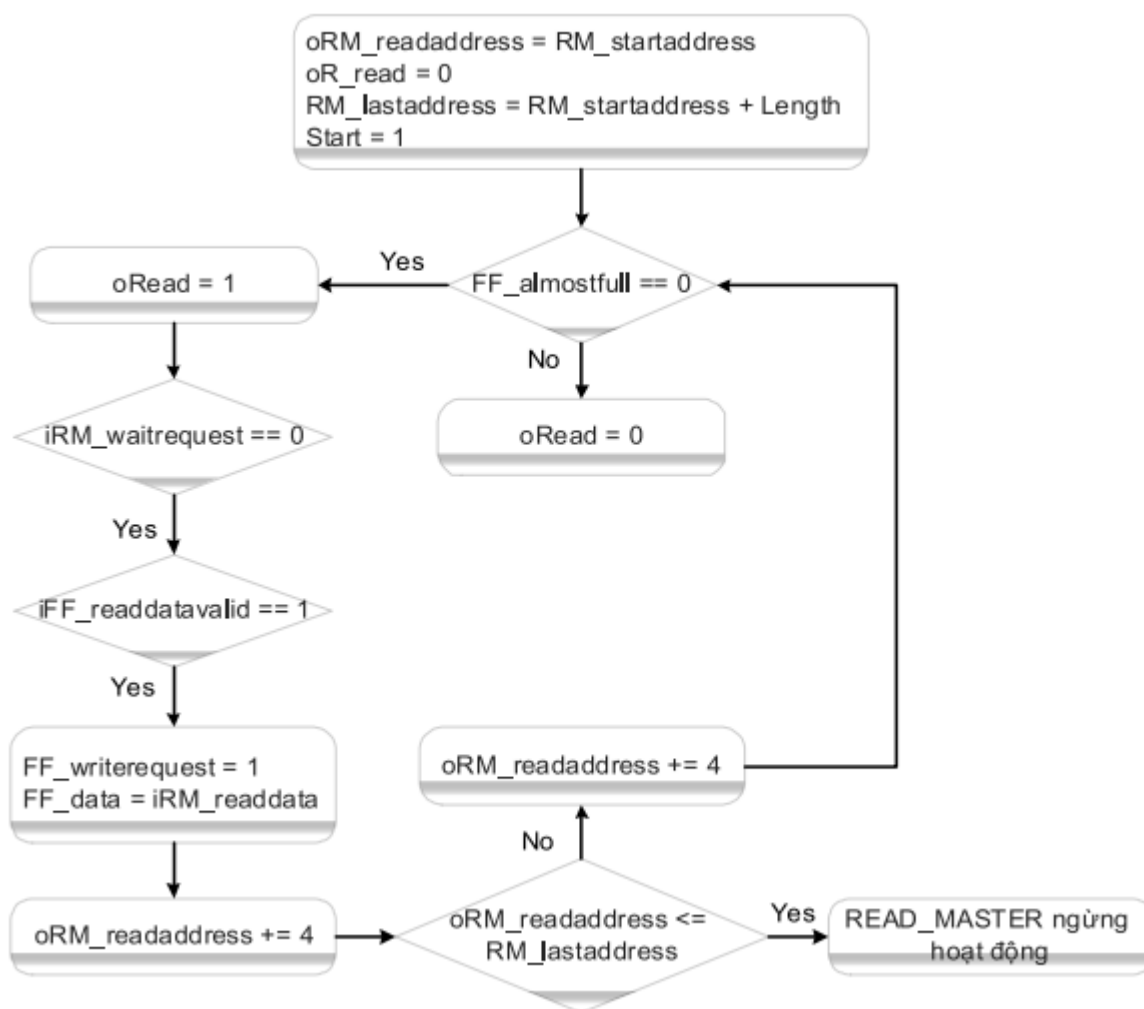
```
assign equal = (pos_write [6:0] - pos_read [6:0]) ? 0:1;
```

sau đó kiểm tra FIFO đã gần đầy hay chưa bằng các dòng code tương ứng sau:

```
assign FF_almostfull = compare & equal;
assign FF_empty = ~compare & equal;
```

Lưu ý: các tín hiệu **FF_almostfull** và **FF_empty** tích cực ở mức cao. FIFO có thể sẽ được báo gần đầy khi lượng dữ liệu đang chứa từ ½ dung lượng của nó trở lên. Bit **equal** bằng 1 khi *pos_read [6:0]* và *pos_write [6:0]* bằng nhau, bit **compare** bằng 1 khi 2bit so sánh khác nhau.

1.2. Thiết kế module read master



Hình 1.2. Sơ đồ giải thuật mô tả hoạt động của Read Master

1.2.1. Lí thuyết và code mô tả verilog

Lí thuyết: nếu có sự thay đổi của bất kì tín hiệu nào từ CPU, sẽ kiểm tra có tín hiệu reset hay không, sau đó kiểm tra tín hiệu `readdatavalid`, nếu có tín hiệu, dữ liệu sẽ chuyển vào thanh ghi **oFF_data**. Sau đó kiểm tra tín hiệu yêu cầu ghi vào FIFO, kiểm tra FIFO có trống hay không. Nếu FIFO trống, kiểm tra tiếp tín hiệu reset có hoạt động hay không, nếu có. Tất cả tín hiệu ra, địa chỉ cuối cùng, thanh ghi state đều phải gán về 0. Nếu reset không hoạt động, thay vào đó là tín hiệu khởi động chạy read master (**iStart**), gán địa chỉ đầu tiên và địa chỉ cuối cùng vào các thanh ghi tương ứng. Tiếp theo đó là quá trình ghi dữ liệu vào FIFO từ các trường

hợp của **state**, tăng địa chỉ lên 4 đơn vị, kiểm tra có trùng với địa chỉ cuối cùng hay không. Cụ thể nguyên lý trên diễn ra như sau:

Đầu tiên, kiểm tra tín hiệu cho phép ghi dữ liệu vào FIFO (**oFF_writerequest**) bằng dòng code sau, nếu **oFF_writerequest** =1 sẽ cho phép ghi vào FIFO:

```
assign oFF_writerequest = (~iReset_n)? 0 : (~iFF_almostfull && ~|state
&& iRM_readdatavalid);
```

Tiếp theo, sử dụng khối always @ (*) để chuyển dữ liệu vào thanh ghi **oFF_data** thông qua việc kiểm tra tín hiệu **readdatavalid**. Nếu tín hiệu reset được cho mức tích cực **oFF_data** = 0;

Tiếp tục sử dụng khối always @(posedge iClk), nếu reset được cho mức tích cực, các thanh ghi sau đều được gán bằng 0 thông qua các dòng code: **state**, **oRM_read**, **oRM_readaddress**, **RM_lastwriteaddress**.

```
if(~iReset_n)
begin
state <= 2'h0;
//oFF_data <= 32'h0;
oRM_readaddress <= 32'h0;
oRM_read <= 1'b0;
RM_lastwriteaddress <= 32'h0;
End
```

Nếu reset không hoạt động, trong khi đó tín hiệu khởi động bộ DMA (**iStart**) tích cực, địa chỉ đọc và địa chỉ cuối cùng của quá trình DMA sẽ được nạp vào các thanh ghi **oRM_readaddress**, **RM_lastwriteaddress** thông qua các dòng code sau:

```
if(iStart)
begin
oRM_readaddress <= iRM_startaddress;
RM_lastwriteaddress <= iRM_startaddress + iLength;
end
```

Trong trường hợp, reset không hoạt động, tín hiệu **iStart** cũng không hoạt động, thy vào đó là tín hiệu báo FIFO và trạng thái state không cùng bằng 0 thì sẽ thực hiện các trường hợp bên dưới:

- state = 0: gán state lên 1;
- state = 1:
 - Nếu địa chỉ đang đọc bằng với địa chỉ cuối cùng cần đọc, state sẽ được gán về lại giá trị 0 thông qua dòng code:

```
if(oRM_readaddress == RM_lastwriteaddress)
    state <= 2'h0;
```

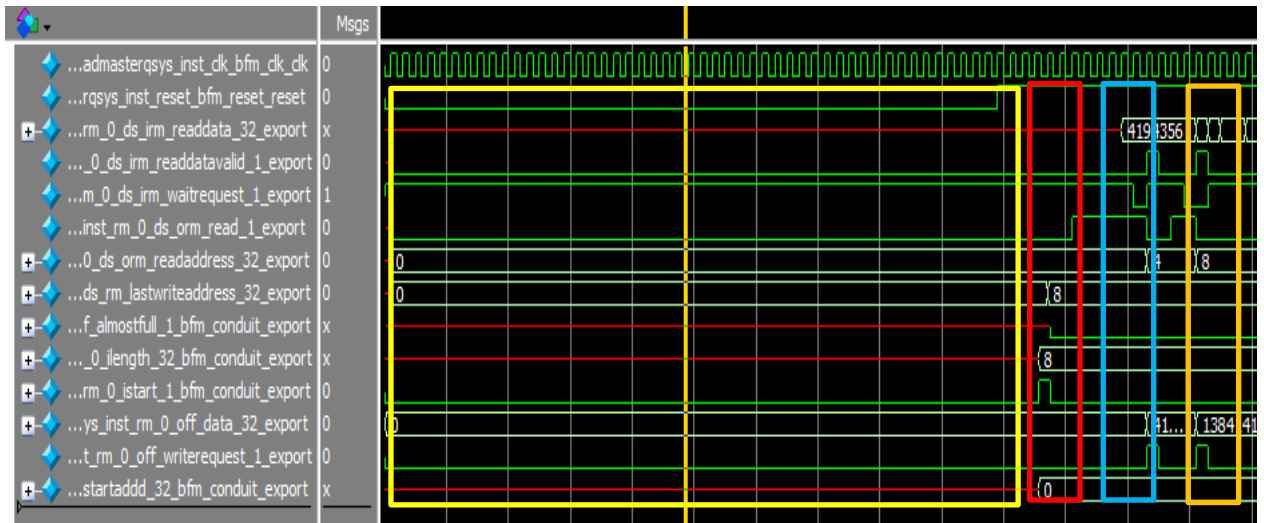
- Nếu không bằng, tín hiệu đọc *oRM_read* sẽ được gọi đi, *state* được gán = 2 với các dòng code:

```
oRM_read <= 1'b1;
state <= 2'h2;
```

- state = 2:
 - Nếu read master không phải chờ tức là tín hiệu **iRM_waitrequest** = 0, địa chỉ đọc hiện tại sẽ được tăng lên 4 đơn vị, tín hiệu yêu cầu đọc **oRM_read** được gán về 0, thanh ghi *state* cũng được gán về 0.

```
if(~iRM_waitrequest)
begin
    oRM_readaddress <= oRM_readaddress + 3'h4;
    oRM_read <= 1'b0;
    state <= 2'h0;
end
```

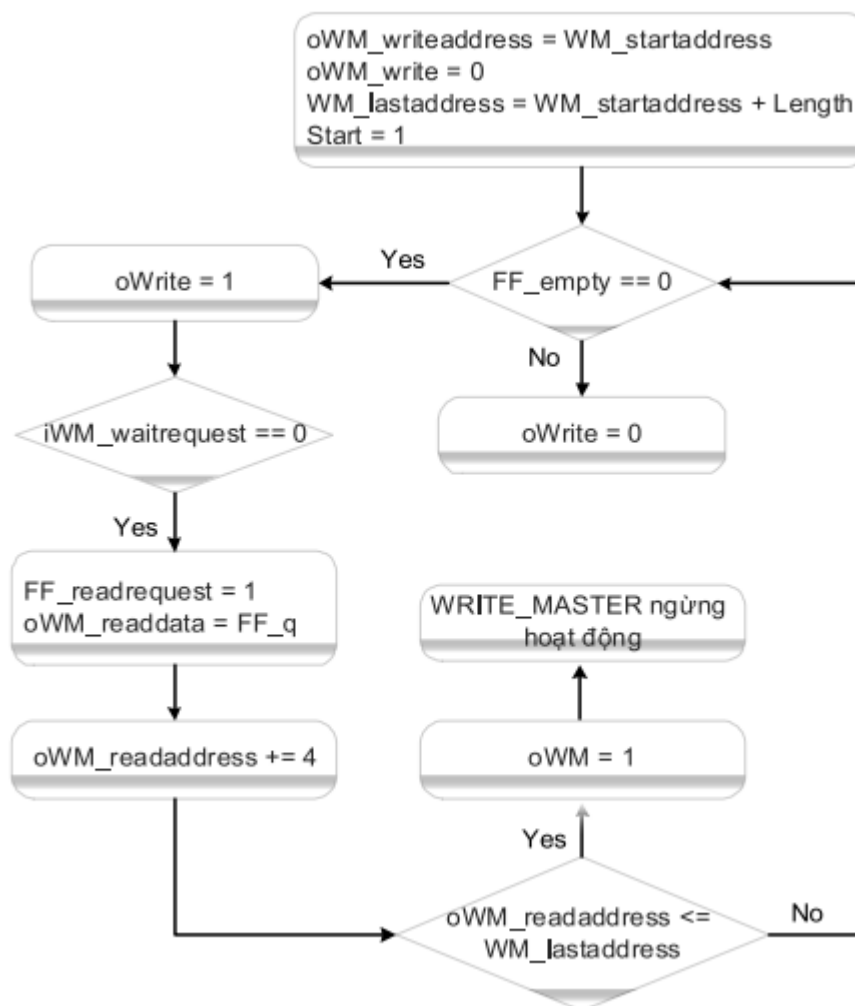
1.2.2. Mô phỏng module read master với Nios II trong Modelsim



Hình 1.3. Kết quả mô phỏng trên phần mềm Modelsim

- Trong vùng của ô màu vàng là giai đoạn reset nên các giá trị là 0 hoặc chưa xác định.
- Trong vùng của ô màu đỏ: **iStartaddress(startadd)** = 1, đồng thời **iStart** = 1, **iLength** = 8, nên **irm_lastwriteaddress** được gán giá trị = 8, sau đó **iStart** được kéo xuống 0, tín hiệu **orm_read** cũng được kéo lên 1, **irm_readdatavalid** = 0, **irm_waitrequest** = 1.
- Trong vùng ô màu lam: tín hiệu waitrequest được kéo xuống 0, tại thời điểm **irm_waitrequest** được kéo lên 1 trở lại, đường địa chỉ **orm_readaddress** được tăng lên = 4, tín hiệu **orm_read** = 0, **irm_readdatavalid** = 1 nên **off_data** = **irm_readdata** = 4194356 (giá trị rác, vì ở đây không khởi tạo giá trị cho mô phỏng), **off_writerequest** = 1 đồng nghĩa cho phép ghi vào fifo.
- Trong vùng ô màu cam: lặp lại quá trình tương tự như vùng ô màu lam ở trên.

1.3. Thiết kế module write master



Hình 1.4. Lưu đồ giải thuật hoạt động của Write Master

1.3.1. Lí thuyết và code mô tả verilog

Nguyên lí: nếu có tín hiệu yêu cầu đọc từ FIFO, sẽ kiểm tra FIFO có đầy hay không. Nếu FIFO đầy, kiểm tra tiếp tín hiệu reset có hoạt động hay không, nếu có. Tất cả tín hiệu ra, địa chỉ cuối cùng, thanh ghi state đều phải gán về 0. Nếu reset không hoạt động, thay vào đó là tín hiệu khởi động chạy write master (`iStart`), gán địa chỉ đầu tiên và địa chỉ cuối cùng vào các thanh ghi tương ứng. Tiếp theo đó là quá trình đọc dữ liệu từ FIFO, tăng địa chỉ lên 4 đơn vị, kiểm tra có trùng với địa chỉ cuối cùng hay không. Nếu trùng, gửi tín hiệu `Done` và kết thúc quá trình DMA. Cụ thể kiểm tra tín hiệu yêu cầu đọc từ FIFO bằng dòng code:

assign oFF_readrequest = (~iReset_n) ? 1'b0 : (~iFF_empty && ~|state);

Tiếp theo đó nếu tín hiệu reset được tích cực, sẽ thực hiện các dòng code sau:

```
if (~iReset_n)
begin
    state <= 2'h0;
    oWM_write <= 1'b0;
    oWM_done <= 1'b0;
    oWM_writedata <= 32'h0;
    oWM_writeaddress <= 32'h0;
    WM_lastwriteaddress <= 32'h0;
end
```

Nếu reset không tích cực mà tín hiệu *iStart* tích cực, các dòng code sau sẽ chạy:

```
if (iStart) //iStart = 1, bắt đầu DMA
begin
    oWM_writeaddress <= iWM_startaddress;
    WM_lastwriteaddress <= iWM_startaddress + iLength;
end
```

Tiếp đến, nếu *iFF_empty* và *state* không đồng thời bằng không, sẽ thực hiện một trong các trường hợp sau:

- *state* = 0: *state* gán lên 1
 - *state* <= 2'h1;
- tín hiệu cho phép xuất dữ liệu khỏi write master sẽ lên 1
 - *oWM_write* <= 1'b1;
- chuyển dữ liệu ra ngoài
 - *oWM_writedata* <= *iFF_q*;
 - *state* = 1;
- Kiểm tra địa chỉ ghi hiện tại có bằng địa chỉ ghi cuối cùng hay không nếu có thì chạy các dòng code sau, **oWM_done=1** tức là quá trình DMA đã kết thúc:
 - *state* <= 2'h2;
 - *oWM_done* <= 1'b1;

- Nếu write master không phải chờ **iWM_waitrequest** =0, tăng địa chỉ ghi lên 4 đơn vị, lệnh xuất dữ liệu bằng 0, **state** gán =0.

if (~iWM_waitrequest)

begin

oWM_writeaddress <= *oWM_writeaddress* + 3'h4;

oWM_write <= 1'b0;

state <= 2'h0;

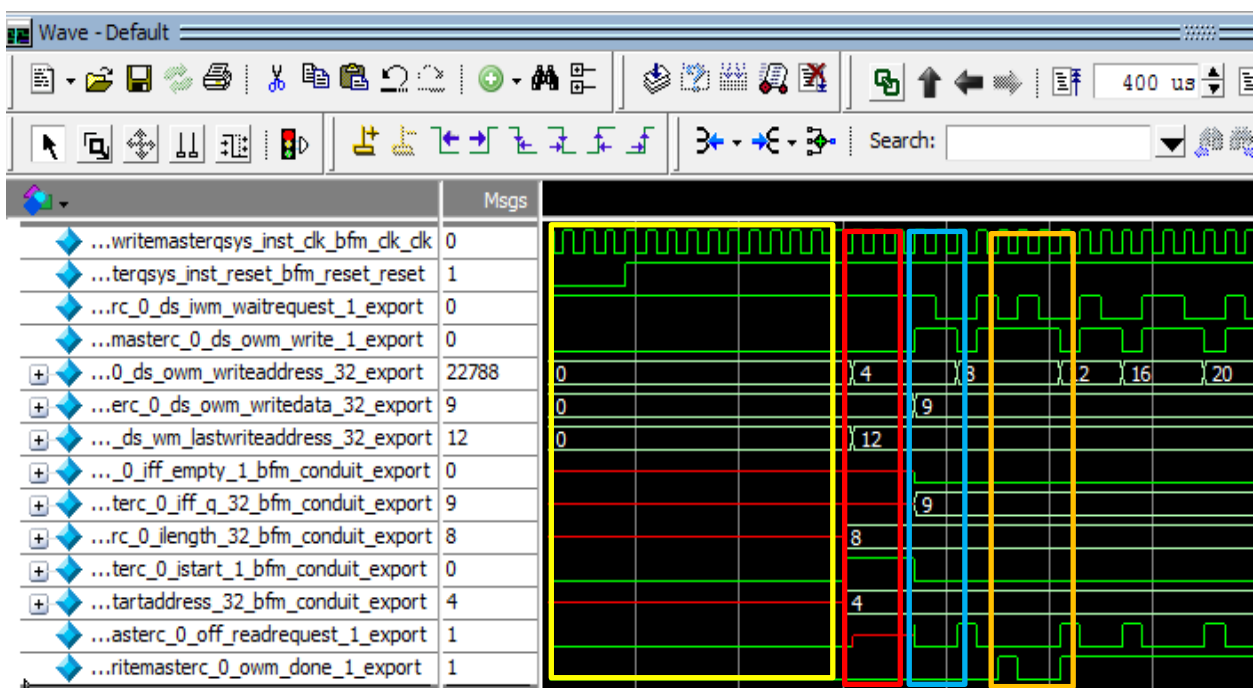
end

- *state* = 2: tín hiệu done (**oWM_done**) và thanh ghi **state** đồng thời được gán về 0.

oWM_done <= 1'b0;

state <= 2'h0;

1.3.2. Mô phỏng module write master với Nios II trong Modelsim



Hình 1.5. Kết quả mô phỏng write master với nios trên modelsim

- Trong ô màu vàng đầu tiên là vùng reset, các tín hiệu = 0 hoặc không xác định

- Ô màu đỏ kế bên: tín hiệu **iStart** lên 1, giá trị của **wm_lastwriteaddress** được gán = **iStartaddress + iLength** và **writeaddress = iStartaddress**.
- Ô thứ 3: tín hiệu write được kéo lên 1, giá trị của **iff_q** được gán cho **owm_writedata**
- Ô thứ 4: tín hiệu **owm_done** được kéo lên 1 báo hiệu quá trình DMA đã kết thúc sau khi **wm_lastwriteaddress = writeaddress + 4**.
- Lưu ý: vì chỉ mô phỏng module write master nên quá trình DMA vẫn chạy dù biến **owm_done** đã bằng 1.

1.4. Thiết kế module control slave.

1.4.1. Code mô tả verilog

Trong khối always @ (posedge iClk) thứ 1 – khối thực hiện việc cấu hình lại các thanh ghi điều khiển của DMA trước khi cho phép DMA hoạt động trở lại, nếu tín hiệu reset hoạt động, các tín hiệu ngõ ra và thanh ghi **control** sẽ được gán về 0.

```

if (~iReset_n)
begin
    oRM_startaddress    <= 32'h0;
    oWM_startaddress    <= 32'h0;
    oLength              <= 32'h0;
    control              <= 32'h0;
    oReaddata            <= 32'h0;
end

```

nếu **reset** không hoạt động, tín hiệu **DONE** bằng 1, thanh ghi **control** sẽ được xóa.

```

else if (DONE)
    control <= 32'h0;

```

nếu cả **reset** và **DONE** cùng không hoạt động nhưng tín hiệu **iChipselect_n** hoạt động, sau đó nếu tín hiệu **iWrite** = 1 và **Busy** = 0, sẽ chạy các lệnh case sau sẽ ghi giá trị của thanh ghi **iAddress** vào các thanh ghi, gồm thanh ghi địa chỉ đọc (**oRM_startaddress**), địa chỉ ghi (**oWM_startaddress**), độ dài (**oLength**), control (**control**). Nếu **iRead** = 1, sẽ đọc trạng thái của thanh ghi **status**.

```

else if (~iChipselect_n)
    begin
        if (iWrite & ~BUSY)
            case (iAddress)
                3'h0: oRM_startaddress <= iWritedata;
                3'h1: oWM_startaddress <= iWritedata;
                3'h2: oLength <= iWritedata;
                3'h3: control <= iWritedata;
            endcase
        else if (iRead)
            case (iAddress)
                3'h7: oReaddata <= status;
            endcase
        end
    end

```

Khởi always thứ 2 sẽ điều khiển việc hoạt động của DMA. Nếu tín hiệu reset hoạt động, các thanh ghi **status**, **oStart**, **state**, **enable** sẽ được đưa về 0. Nếu reset không hoạt động, sẽ thực hiện vòng lặp case với thanh ghi state:

- state = 0: nếu bit **GO** = 1, **enable** = 0, DMA sẽ bắt đầu hoạt động, nạp giá trị cho các bit và thanh ghi **status**, **oStart**, **state**
- state = 1: nạp giá trị cho các bit oStart và enable, đồng thời đợi cho quá trình DMA kết thúc để biến **iMW_done** được đưa lên 1, để nạp giá trị cho thanh ghi **status** và **state** để chuyển sang trường hợp thứ 3.
- state = 2: xóa các thanh ghi **state** và **status** để chuẩn bị cho quá trình DMA mới.

```

always @ (posedge iClk)
begin
    if (~iReset_n)
        begin
            status          <= 32'h0;
            oStart          <= 1'b0;

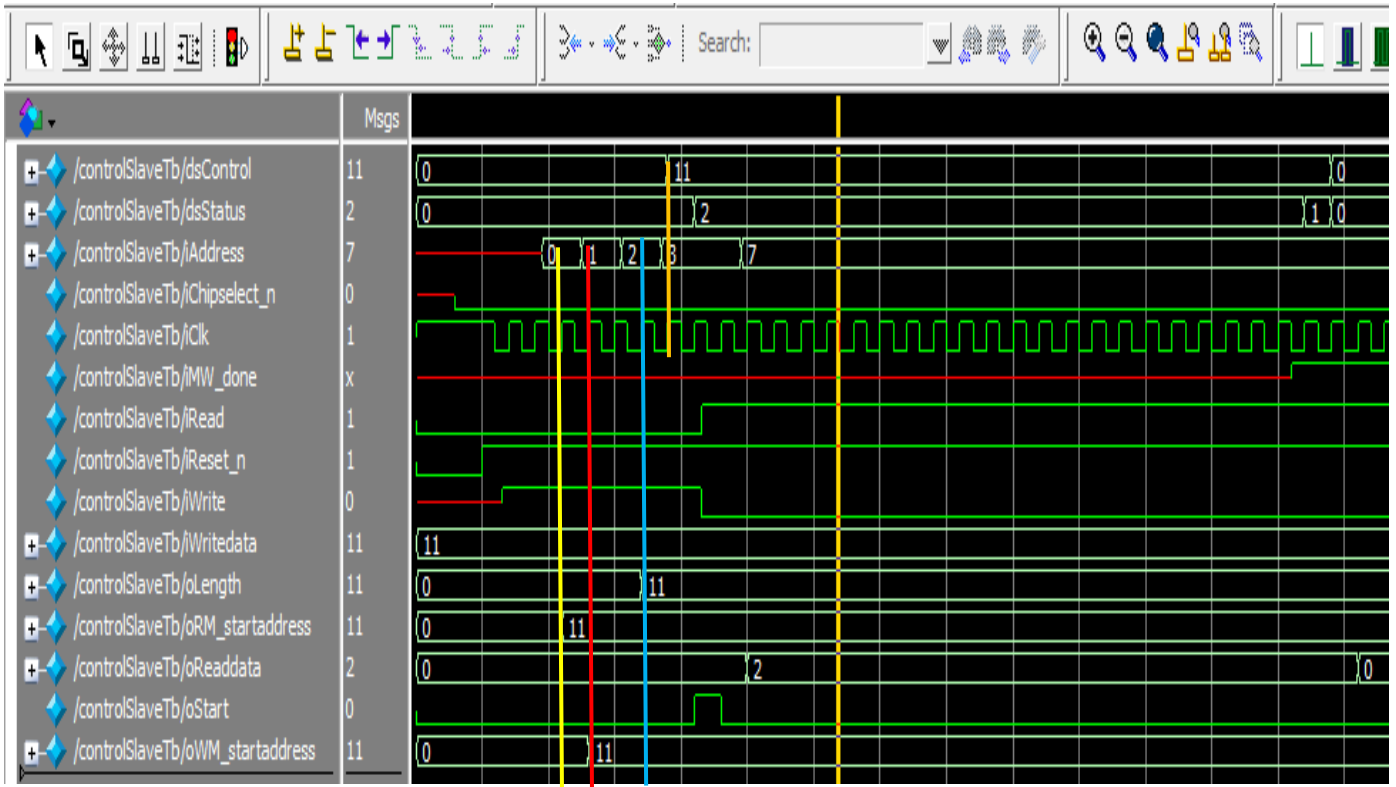
```

```

        state                <= 2'h0;
        enable                <= 1'b0;
    end
    else
        case (state)
            2'h0: begin
                if (GO && ~enable)
                    begin
                        status    <= 32'h2;
                        oStart    <= 1'b1;
                        state     <= 2'h1;
                    end
                end
            end
            2'h1: begin
                oStart          <= 1'b0;
                enable          <= 1'b1;
                if (iMW_done)
                    begin
                        status    <= 32'h1;
                        state     <= 2'h2;
                    end
                end
            end
            2'h2: begin
                status          <= 32'h0;
                state           <= 2'h0;
            end
        endcase
    end
end

```

1.4.2. Mô phỏng module control slave trên modelsim (không sử dụng nios)



Hình 1.6. Kết quả mô phỏng module control slave trên modelsim

- Tại vạch màu vàng nạp giá trị cho **oRM_startaddress**
- Tại vạch màu đỏ nạp giá trị cho **oWM_startaddress**
- Tại vạch màu lam nạp giá trị cho **oLength**
- Tại vạch màu cam nạp giá trị cho **control** (dsControl)
- Tiếp đến thanh ghi **status** được nạp giá trị = 2 (32'h02) vì bit **GO** = 1 (4bit cuối của control = 1011).
- **iMW_done** = 1 dẫn đến **status** = 1, sau đó cả **control** và **status** đều bị xóa về 0, DMA ngừng chạy.

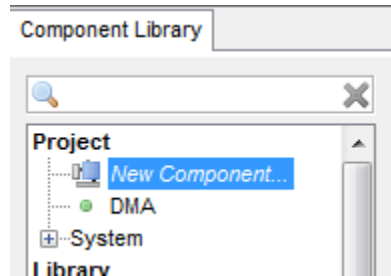
⇒ module đã hoạt động đúng theo ý muốn.

Sau khi thiết kế hoàn tất 4 module trên, tạo file verilog gọi tất cả 4 module trên tạo thành 1 file thống nhất. (file này chính là file dùng để thêm component mới vào ở trong qsys).

2. Thêm khối DMA tự tạo trên vào Qsys, tạo hệ thống, biên dịch, chạy trên board thực tế

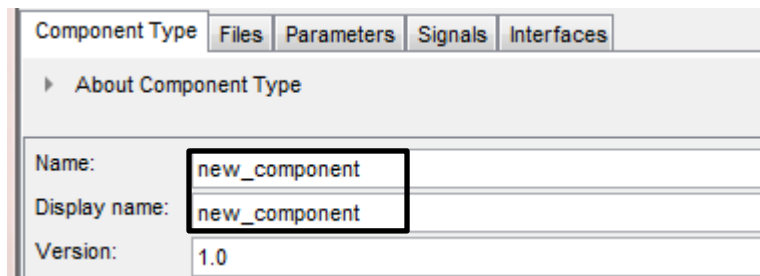
2.1. Thêm DMA vào qsys

- Mở Qsys lên, chuột trái 2 lần vào “*New Component*”



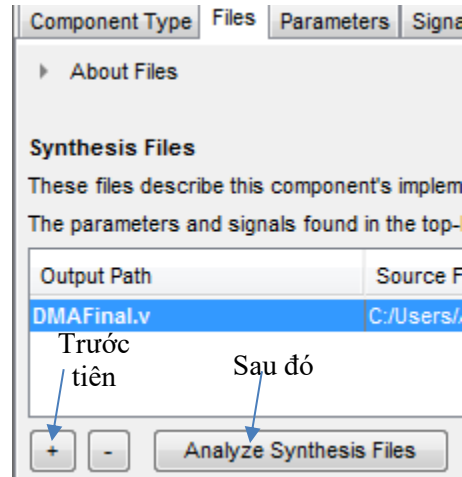
Hình 2.1. Component Library trong Qsys

- Tại thẻ Component type, gõ tên mong muốn vào 2 trường **Name** và **Display name**.



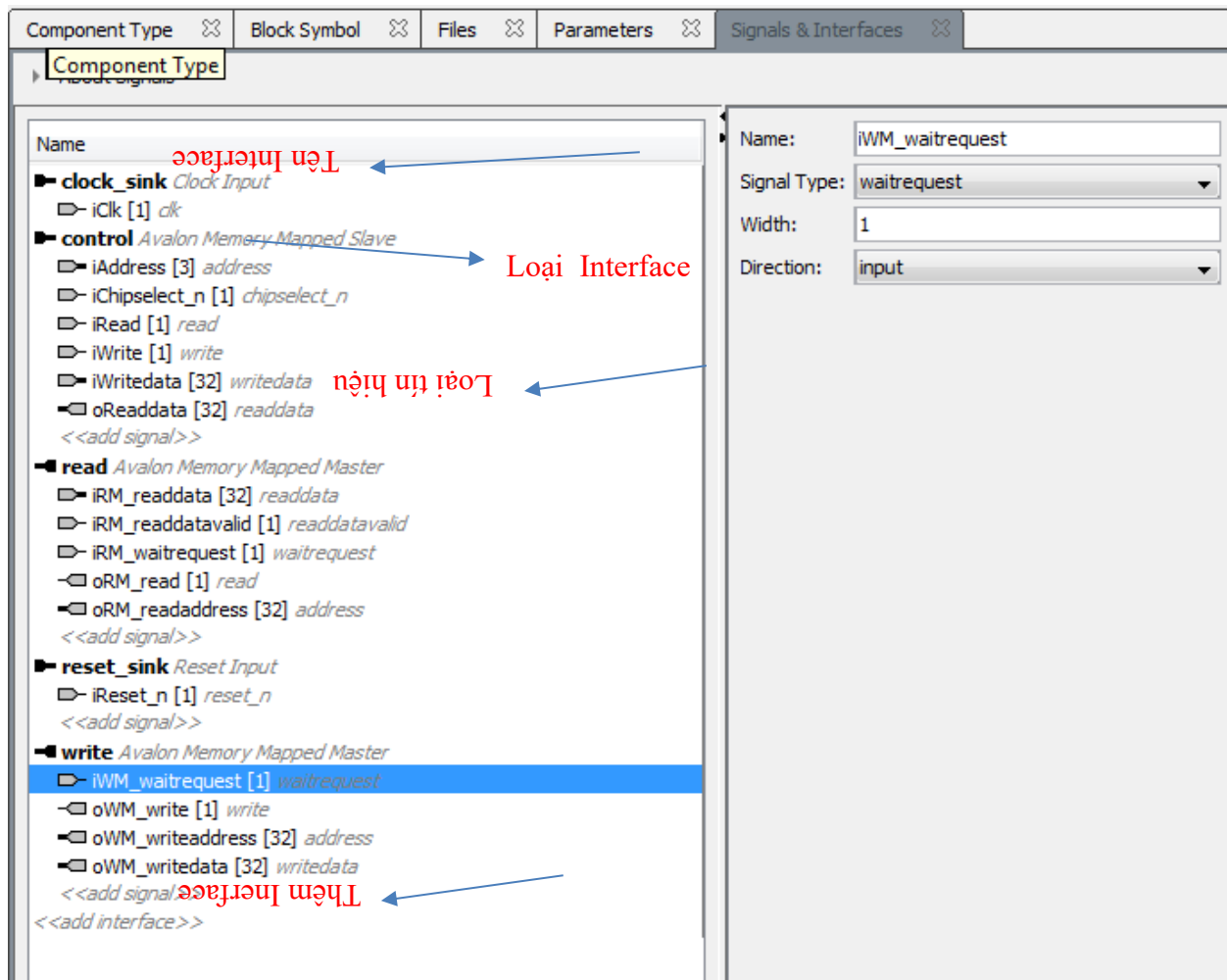
Hình 2.2. Thêm tên vào cho component mới

- Chuyển sang thẻ **Files**, ấn vào dấu “+” trong phần **Synthesis Files** chọn file đã tạo để gọi 4 modules đã viết ở trên (DMAFinal.v trong các file.v đính kèm), sau đó ấn **Analyze Synthesis Files**, nếu không báo lỗi chuyển sang bước tiếp theo.



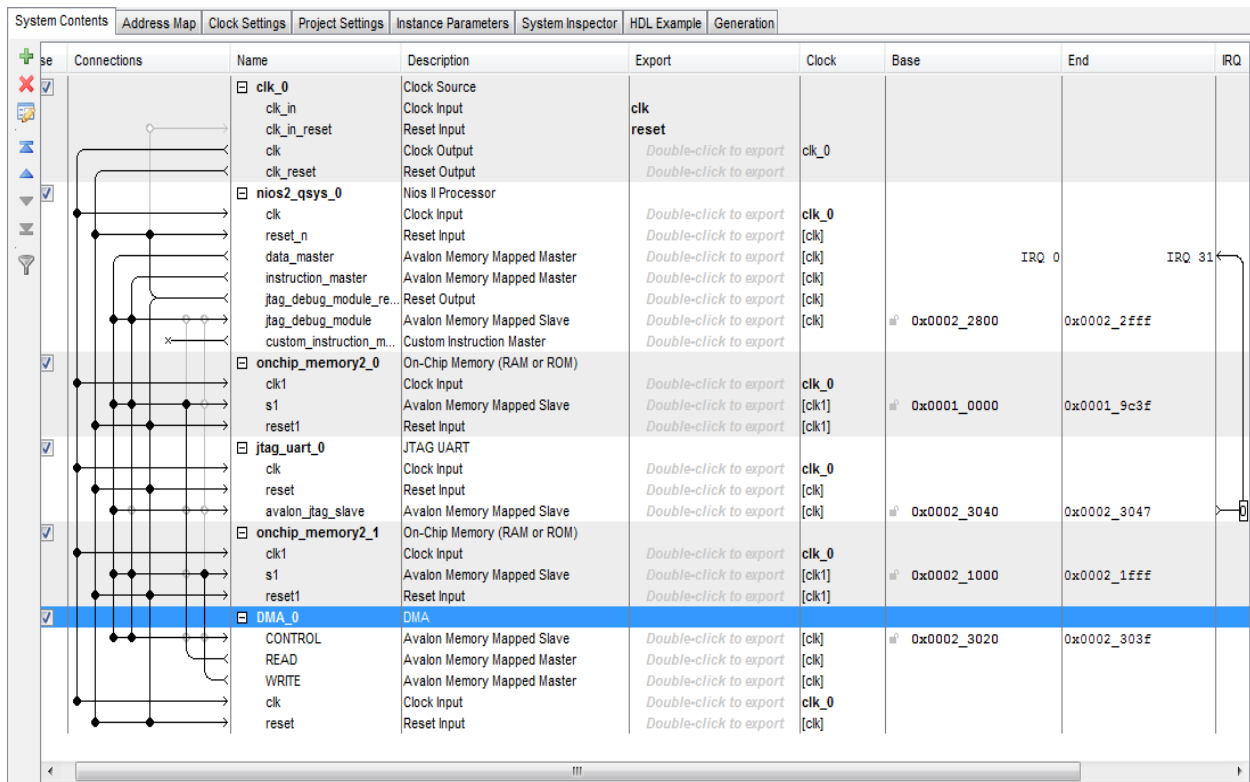
Hình 2.3. Chọn file để nhúng vào trong Qsys

- Chuyển sang thẻ **Signals&Interfaces**
 - Tạo 5 Interfaces 1 Avalon Memory Mapped Slave, 2 Avalon Memory Mapped Master, 1 Clock Input và 1 Reset Input bằng cách chuột trái vào <<add interface>>, tiếp đến click vào tên mỗi interface, chọn Associated Clock và Associated Reset cho mỗi interface.
 - Kéo thả các tín hiệu vào interface và chọn loại tín hiệu (Signal type) cho mỗi tín hiệu như hình bên dưới.



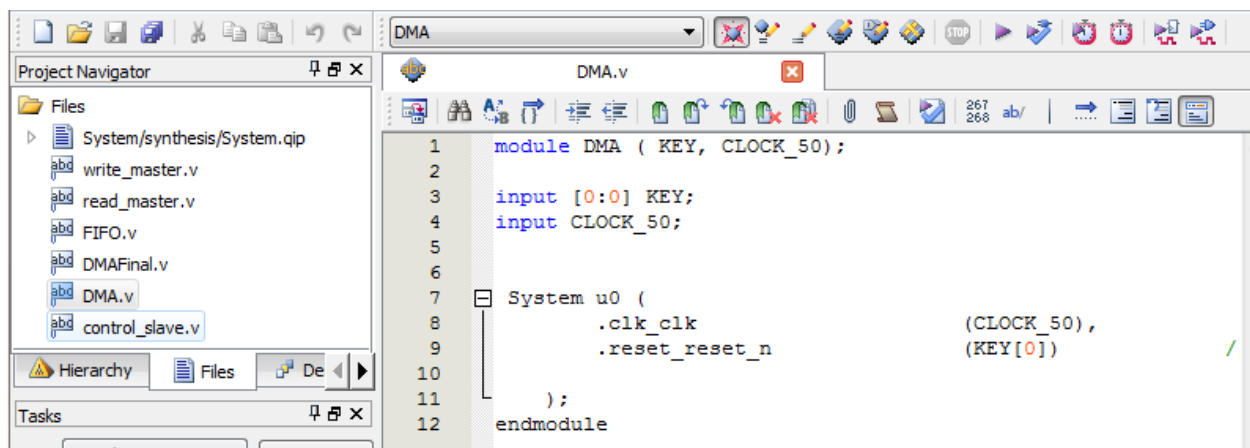
Hình 2.4. Các Interface và loại tín hiệu cần có

2.2. Tạo hệ thống sử dụng bộ DMA vừa tạo như hình bên dưới



Hình 2.5. Các thành phần của hệ thống và cách kết nối chúng trong Qsys

2.2.1. Biên dịch, nạp file .sof xuống board



Hình 2.6. File top và file cần thiết để biên dịch hệ thống

2.2.2. Mở Eclipse, tạo file project mới với file .sopcinfo, tạo file.c để kiểm tra

Trong file .c được sử dụng ở đây gồm:

- Thêm vào các thư viện, io.h, system.h, stdio.h
- Khai báo biến length, biến control và định nghĩa DMA
- Khởi tạo 2 vùng bộ nhớ pdata0 (chứa dữ liệu cần sao chép), pdata1 (nơi sẽ lưu dữ liệu vào).
- Hàm cấu hình cho DMA trước khi nó hoạt động, gồm có:
 - Cấu hình địa chỉ bắt đầu đọc:
 $IOWR_32DIRECT(DMA, 0*4, (int)pdata0);$
 - Cấu hình địa chỉ bắt đầu ghi:
 $IOWR_32DIRECT(DMA, 1*4, (int)pdata1);$
 - Cấu hình độ lớn dữ liệu cần sao chép
 $IOWR_32DIRECT(DMA, 2*4, length);$
 - Cấu hình thanh ghi control
 $IOWR_32DIRECT(DMA, 3*4, control);$
- Hàm main: Xử dụng polling để kiểm tra khi nào DMA hoàn thành và sau đó in các giá trị theo con trỏ pdata1